Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:04'te başlar.
- Önceki dersin tekrarı
  - Ön tasarım ve ayrıntılı tasarım.
  - İşlevsel çözümlemede yapı diyagramı, fiziksel veri akış diyagramları ve HIPO.
- Nesneye dayalı tasarım modelinin oluşturulması
  - Analiz "ne?" sorusunu, tasarım "nasıl?" sorusunu yanıtlar.
  - Kavramsal sınıflar yazılım sınıflarına dönüşür ve sorumluluklar (metotlar) atanır.
- Etkileşim diyagramları
  - İşbirliği diyagramı (collaboration diagram) ve ardışıl diyagram (sequence diagram).
  - Mesaj numaralandırma, kendine mesaj, koşullu mesaj, karşılıklı dışlamalı mesaj, döngü.
  - Çizim araçları: Violet UML, ArgoUML.
- Kullanım senaryolarının gerçeklenmesi (use case realization)
  - Bilme ve yapma sorumlulukları.
  - İteratif çalışma: önce ana başarılı akış, sonra alternatif akışlar.
- GRASP: 9 genel sorumluluk atama prensibi
  - Uzman, Yaratıcı, Az Bağımlılık, İyi Uyum, Denetçi, Çok Şekillilik, Yapay Sınıf, Dolaylılık, Değişimlerden Korunma.
  - Ek: "Yabancılarla konuşma" prensibi.
- İlklendirme (başlangıç) senaryosu, görünürlük (visibility) türleri.
- Tasarım sınıf diyagramı ve kodlamaya geçiş.

Hocanın Özellikle Vurguladığı Kısımlar

- Tasarımda bütün iş, hangi sorumluluğun hangi sınıfa verileceğine karar vermektir.
  - "Kim neden sorumlu olsun?" sorusu tasarımın özüdür.
- Tekerleği yeniden icat etmeyin. Daha önce iyi çalıştığı bilinen çözümleri, yani tasarım kalıplarını kullanın.
- İki etkileşim diyagramı birbirinin alternatifidir. İkisi de aynı bilgiyi verir.
  - Genellikle ardışıl diyagram tercih edilir, çünkü mesaj sırası daha kolay okunur.
- Ardışıl diyagramlar bazı kaynaklarda analizde de kullanılır. Ama asıl kullanım amacı tasarımda sınıfların iş birliğini göstermektir.
- Sorumluluk atarken 1-4 numaralı GRASP prensipleri (Uzman, Yaratıcı, Az Bağımlılık, İyi Uyum) birlikte düşünülmelidir.
- Yapay sınıfları gereğinden fazla kullanmayın. Fazla kullanılırsa kavramsal çözümlemeden işlevsel çözümlemeye kayılır.
- Adaptör ve ara katman koyun ama abartmayın. Katman üstüne katman yapmayın, bir adaptörle çözün.
- Kodlamaya en az bağımlı sınıftan başlayın, en çok bağımlı sınıfa doğru gidin. Her sınıfı yazdıkça birim testine tabi tutun.

Kısa Tekrar Notları

- Tasarım modeli = mantıksal çözüm. Gösterimi:
  - Dinamik yapı: etkileşim diyagramları (işbirliği veya ardışıl).

  - Statik yapı: tasarım sınıf diyagramı (design class diagram, software class diagram).
- İşbirliği diyagramı: nesneler düzlemde, mesajlar 1, 1.1, 1.2, 2 diye numaralı. Az yer kaplar, sıra takibi zordur.
- Ardışıl diyagram: her nesnenin yukarıdan aşağı yaşam çizgisi var. Metot çalışması kalın çubukla gösterilir. Daha çok yer kaplar, sıra kolay okunur.
- Gösterimler:
  - Nesne: `n1 : Nesne` (örnek adı : sınıf adı).
  - Dönüş değeri: kesikli ok (genellikle çizilmez).
  - Kendine mesaj: nesne kendi metodunu çağırır (`self.mesaj`).
  - Koşullu mesaj: `[koşul]`. Ardışıl diyagramda `opt` kutusu.
  - Karşılıklı dışlamalı (if-else): işbirliğinde 1a/1b, ardışıl diyagramda kesikli çizgiyle bölünmüş `alt` kutusu.
  - Döngü: işbirliğinde * (her eleman için), ardışıl diyagramda `loop` kutusu. Koşul `[i = 1..n]` gibi yazılır.
  - Nesne yaratma: `create` mesajı. Ardışıl diyagramda yeni nesne, yaratıldığı yükseklikte başlar.
- Sorumluluk türleri:
  - Bilme: kendi verileri, ilişkili nesneler, hesaplanabilen (türetilen) veriler.
  - Yapma: nesne yaratma/yok etme, başka nesneyi tetikleme, işi başkasına devretme, başka nesneleri denetleme.
- GRASP (General Responsibility Assignment Software Patterns):
  1. Uzman (Information Expert): sorumluluğu, gerekli bilgiye sahip sınıfa ver.
  2. Yaratıcı (Creator): B, A'yı içeriyorsa, kaydını tutuyorsa, yakından kullanıyorsa veya A'nın başlangıç verilerine sahipse A'yı B yaratsın.
  3. Az Bağımlılık (Low Coupling): sorumlulukları bağımlılığı en aza indirecek şekilde ata.
  4. İyi Uyum (High Cohesion): bir sınıfın sorumlulukları birbiriyle ilgili ve odaklı olsun.
  5. Denetçi (Controller): sistem olaylarını bir ön yüz (façade) denetçisi veya senaryo (use case/session) denetçisi karşılasın.
  6. Çok Şekillilik (Polymorphism): tipe göre değişen davranışları if-else yerine polimorfik metotlarla gerçekle.
  7. Yapay Sınıf (Pure Fabrication): gerçekte olmayan bir sınıf yarat (ör. veritabanı işlemleri sınıfı).
  8. Dolaylılık (Indirection): değişken iki birim arasına ara birim nesnesi koy (adaptör).
  9. Değişimlerden Korunma (Protected Variations): değişime açık noktaların etrafında kararlı arayüzler oluştur.
- GoF (Gang of Four) kalıpları: toplam 23 tane.
- Bağımlılık nedenleri: B tipinde üye, B'nin metodunu çağırmak, B tipinde parametre veya yerel değişken, B'den türemek.
- Görünürlük: özellik (attribute), parametre, yerel (local), global.
- Toplam tutar örneği: 3 kalem × 10 TL + 5 silgi × 5 TL + 2 kalemtıraş × 20 TL = 95 TL.

Detaylı Açıklamalar

Nesneye dayalı tasarım modeli

Analizde kavramlar, yapılması gerekenler ve sorumluluklar bulundu. Tasarımda bunların nasıl yapılacağına karar verilir. Kavramsal sınıflar mümkün olduğunca gerçek dünyaya benzer şekilde yazılım uzayına taşınır. Bunlara davranışlar (metotlar) eklenir ve yazılım sınıfı hâline gelirler.

Tasarımın girdileri kullanım senaryoları (use case) ve sözleşmelerdir (contract). Sözleşmeler bu derste ayrıntılı işlenmedi. Senaryolardaki fiiller yapılması gereken işleri, yani sorumlulukları gösterir. Sorumluluklar tasarım prensipleri ve kalıpları ışığında sınıflara atanır. Sonuç etkileşim diyagramları ve tasarım sınıf diyagramıyla gösterilir.

Bu derste UML'in yalnızca iki diyagram türü tanıtıldı. Diğer diyagramlar ileri seçmeli derslere bırakıldı.

İşbirliği ve ardışıl diyagram

Slayttaki örnekte `n1 : Nesne` nesnesine bir mesaj gelir. Bu metot çalışırken sırayla başka nesnelerin metotlarını çağırır:

- 1: n1'e gelen ilk çağrı.
- 1.1: n1, n2'deki bir metodu çağırır.
- 1.2: n1, n3'teki bir metodu çağırır. Bu metot çalışırken n3, n2'deki başka bir metodu çağırır.
- 2: n1, n4'teki bir metodu çağırır.

İşbirliği diyagramında bu sıra yalnız numaralardan okunur. Ardışıl diyagramda ise her çağrı, çağıranın aktivasyon çubuğu içinden çıkan bir ok olarak görünür. İç içe çağrılar iç içe çubuklar olarak görülür.

Dönüş değeri gerekiyorsa kesikli bir okla gösterilir. Normalde çizilmez.

Bir nesne kendi başka bir metodunu da çağırabilir. Ardışıl diyagramda bu, metot gövdesinin içinden aynı nesneye dönen bir okla gösterilir.

Koşullu mesaj: Köşeli parantez içinde koşul yazılır. Örnek: `[not ortalaması > 3]`. Ardışıl diyagramda mesaj `opt` kutusu içine alınır.

Karşılıklı dışlamalı mesajlar (if-else): İşbirliği diyagramında 1a ve 1b diye iki dal olur. Koşul doğruysa 1a (n2'deki metot), yanlışsa 1b (n3'teki metot) çalışır. 1b'nin içinde 1b.1 ile n3, n4'ü çağırır. Ardışıl diyagramda `alt` kutusu kullanılır ve kutu kesikli çizgiyle ikiye bölünür. Hoca çizimde bir hata yaptı ve derste düzeltti: 1b.1 mesajı n1'den değil, n3'ten n4'e gitmelidir.

Döngü: Bir koleksiyonun bütün elemanları için çağrı yapılacaksa işbirliği diyagramında * kullanılır. Örnek: bütün sipariş kalemleri için `altToplamAl()` çağrılır. Bu bir for-each döngüsüdür. Ardışıl diyagramda `loop` kutusu kullanılır. Eski UML 1 sürümlerinde koleksiyon, arka arkaya üst üste konmuş kutularla gösterilirdi.

Araçlar: Violet UML hafif ve ücretsiz, Java tabanlı bir çizim aracıdır. ArgoUML biraz daha gelişmiştir. Daha profesyonel araçlar ve Eclipse eklentileri de vardır.

Ardışıl diyagramın başında bazen bir aktör çizilir. Örnek: kasiyer ekranda "satışı tamamla" butonuna tıklar ve bu, ilk nesnenin metodunu tetikler.

Kullanım senaryolarının gerçeklenmesi

Tasarım, yazılım sınıflarına metotları eklemek ve senaryoları yerine getirmek için nesneler arasında hangi sırayla hangi mesajların gideceğini belirlemektir.

Sorumluluk türleri (öğrenci örneği):

- Bilmesi gerekenler:
  - Kendi verileri: ad, soyad, numara, sınıf, bölüm.
  - İlişkili nesneler: aldığı derslerin listesi.
  - Hesaplanabilen veriler: notlardan hesaplanan not ortalaması.
- Yapması gerekenler: başka nesne yaratmak veya yok etmek, başka nesneyi tetiklemek, işi başkasına devretmek, başka nesneleri denetlemek.

Sorumluluklar metotlar olarak sınıflara verilir. Bir senaryonun gerçeklenmesi tek bir metotla bitmez. Metot başka nesnelerin metotlarıyla iş birliği yapar.

Bu iş iteratif yapılır:

1. En basit senaryodan başlanır.
2. Önce ana başarılı akış (main success flow) gerçeklenir.
3. Sonraki iterasyonlarda alternatif akışlar eklenir (hata durumu, nakit veya kartla ödeme vb.).

Bütün senaryolar bitince kodlanmaya hazır bir çözüm taslağı, yani tasarım elde edilir. Bu sırada analizde atlanan kavramlar da ortaya çıkabilir. Bunlar geri dönülüp kavramsal modele eklenir. Analiz modeline metot eklenmez, yalnız yeni kavramlar eklenir.

Satış örneği ve Uzman kalıbı

Senaryo: Müşteri ürünlerle kasaya gelir. Kasiyer yeni satış başlatır ve ürün kimliğini girer. Sistem satışı bir satır olarak kaydeder, ürün tanımını, fiyatını ve toplam tutarı gösterir. Sonunda vergilerle birlikte toplam gösterilir.

"Satışın toplam bedelini kim hesaplamalı?" sorusu Uzman kalıbıyla yanıtlanır:

- Önce yazılım sınıflarına bakılır. Uygun sınıf yoksa kavramsal sınıflara bakılır. En uygunu yazılım sınıfı olarak modele alınır. Böylece gerçek dünya ile yazılım arasındaki temsil farkı azalır (low representational gap).
- Bütün satış satırlarını bilen `Sale` sınıfıdır. `getTotal()` metodu ona verilir.
- Her satır kendi alt toplamını bilir. `SalesLineItem` sınıfına `getSubtotal()` verilir.
- Bir ürünün birim fiyatını ürün tanımı bilir. `ProductSpecification` sınıfına `getPrice()` verilir. Hoca bu bölümde sınıfı "product description" diye okudu (00:48:56, 00:50:52); aynı kavramdır.

Çalışma sırası:

1. `t = sale.getTotal()` çağrılır.
2. `Sale`, içindeki bütün satırlar için `getSubtotal()` çağırır (döngü).
3. Her `SalesLineItem`, `ProductSpecification`'dan `getPrice()` ile birim fiyatı alır ve miktarla çarpar.
4. `Sale` alt toplamları toplar ve sonucu döndürür.

Satır	Miktar	Birim fiyat	Alt toplam
Kalem	3	10 TL	30 TL
Silgi	5	5 TL	25 TL
Kalemtıraş	2	20 TL	40 TL
Toplam			**95 TL**

Toplam Python ile doğrulandı.

```
// getTotal()'ın tasarımdan çıkan iskeleti
public Money getTotal() {
    Money toplam = 0;
    for (SalesLineItem satir : satirlar) {
        toplam += satir.getSubtotal(); // içinde: miktar * urunTanimi.getPrice()
    }
    return toplam;
}
```

Not: Kod hocanın sözlü anlatımından kurulan bir iskelettir. `Money` tipi gerçek kodda bir sınıf olacağı için `+=` doğrudan çalışmaz; yalnız fikri gösterir.

Yaratıcı (Creator)

Soru: Bir sınıftan nesne yaratma sorumluluğu kimde olmalı?

Cevap: B, şu durumlardan biri varsa A nesnelerini yaratsın:

- B, A nesnelerini içeriyorsa.
- B, A'ların kaydını tutuyorsa.
- B, A'ları yakından kullanıyorsa.
- B, A'nın yaratılması için gereken başlangıç verilerine sahipse.

Örnek: Kasiyer "3 ×" yazıp barkodu okutur. `Register`, `Sale` nesnesine `makeLineItem(spec, 3)` çağrısı yapar. Satırları `Sale` tuttuğu için `SalesLineItem`'ı `Sale` yaratır (`create`). Ardışıl diyagramda `SalesLineItem` nesnesi en üstte değil, yaratıldığı noktada başlar.

Az Bağımlılık ve İyi Uyum

Az Bağımlılık (Low Coupling): Bir sınıf, iç işleri için başka sınıfları ne kadar az kullanır ve onlar hakkında ne kadar az bilirse o kadar iyidir. Bağımlılık artarsa bir sınıftaki değişiklik bağlı sınıfları da etkiler. Sınıfları anlamak ve yeniden kullanmak zorlaşır.

A, B'ye şu durumlarda bağımlıdır:

- A'nın B tipinde bir üyesi varsa.
- A, B'nin bir metodunu çağırıyorsa.
- A'nın bir metodunda B tipinde parametre veya yerel değişken varsa (yalnız metot süresince).
- A doğrudan veya dolaylı olarak B'den türemişse.

İyi Uyum (High Cohesion): Bir sınıfın sorumlulukları birbiriyle ilgili ve belli bir konuya odaklı olmalıdır. Birbiriyle ilgisiz çok iş yapan sınıfın uyumu kötüdür. Bunun uç noktası her şeyin tek sınıfta toplandığı "god class"tır. Böyle sınıfları anlamak, bakımını yapmak ve yeniden kullanmak zordur. Değişimlerden çok etkilenirler.

İyi uyumdan bazen bilerek vazgeçilir:

- Bütün veritabanı işlerini tek bir yapay sınıfa toplamak. Bu işi ayrı bir ekibe vermeyi kolaylaştırır.
- Dağıtık sistemde ağ trafiğini azaltmak için daha büyük ve daha az uyumlu nesneler kullanmak.

Ödeme örneği:

- İlk tasarım: `makePayment` mesajı `Register`'a gelir. `Register` bir `Payment` nesnesi yaratır ve `sale.addPayment(p)` ile satışa bağlar. Gerçek hayatı yansıtır ama `Register`, `Payment`'a gereksiz yere bağımlı olur ve uyumu bozulur.
- Daha iyi tasarım: `Register` işi doğrudan `sale.makePayment(...)` diye satışa devreder. `Payment`'ı onu en çok kullanacak olan `Sale` yaratır. Böylece `Register`'ın bağımlılığı azalır ve uyumu korunur.

Denetçi (Controller)

Soru: Butona tıklama gibi sistem olaylarını kim karşılasın?

Cevap: İki seçenek vardır:

- Bütün sistemi veya alt sistemi temsil eden bir ön yüz denetçisi (façade controller).
- Her senaryo için ayrı bir senaryo veya oturum denetçisi (use case / session controller). Olay sayısı çok fazlaysa ve uyum bozuluyorsa bu tercih edilir.

Denetçi işi kendisi yapmaz, ilgili nesneye aktarır. Faydaları:

- Arayüz katmanı ile iş mantığı katmanı birbirinden ayrılır. İkisi bağımsız geliştirilip değiştirilebilir.
- Görevi doğrudan bir pencere (window) nesnesine verirseniz yeniden kullanılabilirlik ve esneklik kaybolur.
- İş sırası kontrolleri de denetçide yapılabilir. Örnek: sipariş vermeden ödemeye basılmasını engellemek.

Örnek: Kasiyer ekranda barkodu ve miktarı girer, "Enter Item" butonuna basar. Bu olay bir `JFrame`'de oluşur (`actionPerformed`). Olayı alan ve iş katmanına aktaran nesne denetçidir; örnekte `Register`. Arayüz katmanı ile iş katmanı arasında bir tampon bölge oluşur. Yarın mobil arayüz veya self-servis kiosk eklense de iş katmanı değişmez. Gerekirse yalnız denetçide küçük değişiklikler yapılır.

Çok Şekillilik (Polymorphism)

Soru: Tipe bağlı alternatif davranışlar nasıl ele alınmalı? Sisteme kolayca takılıp çıkarılabilen parçalar nasıl tasarlanmalı?

Cevap: Davranış sınıfa göre değişiyorsa if-else yerine polimorfik metotlar kullanın.

Örnekler:

- Vergi hesabı: `Sale`, ortak bir arayüz üzerinden `getTaxes()` çağırır. Arkada hangi vergi sistemi adaptörünün çalıştığını bilmez.
- Kredi kartı ödemesi: Visa, MasterCard, Troy, American Express gibi seçenekler eklenip çıkarılabilir. Uygulama için önemli olan yalnız kartla ödeme almaktır.

Gerçek çözümler birkaç GoF kalıbının birlikte kullanılmasını gerektirir. Bu derste bu kadarını bilmek yeterli.

Yapay Sınıf (Pure Fabrication)

Uzman kalıbının önerdiği çözüm, Az Bağımlılık ve İyi Uyum ile çelişebilir. O zaman gerçekte olmayan bir sınıf yaratılır. Örnek: bağlantı açma, okuma, yazma, ekleme, güncelleme, silme gibi bütün veritabanı işlerini yapan tek bir sınıf. Bütün kavramlar veritabanına ayrı ayrı bağlanmaz, yalnız bu aracıyla konuşur. Uyum artar, bağımlılık ve karmaşıklık azalır.

Yapay sınıflar fazla kullanılırsa "veritabanı işlemleri sınıfı, ödeme işlemleri sınıfı, raporlama işlemleri sınıfı…" diye işlevsel çözümlemeye geri dönülür. Gerçek dünyanın yazılıma taşınmasından uzaklaşılır ve bağımlılıklar artar. Yalnız gerektiği yerde, bir ara katman veya adaptör mantığıyla kullanılmalıdır.

Dolaylılık ve Değişimlerden Korunma

Dolaylılık (Indirection): Kolay değişebilen iki birim arasındaki bağımlılığı azaltmak için araya bir ara birim nesnesi konur. Denetçi ve veritabanı sınıfı da birer ara birimdir. GoF'taki Adapter, Façade ve Observer gibi kalıplar bu prensibin özelleşmiş hâlleridir. Örnek: `Sale`, vergi listesini bir `TaxMasterAdapter` nesnesinden ister. Adaptör dış sistemle TCP soket veya web servisiyle konuşabilir; bu `Sale`'i ilgilendirmez. Dış sistem değişirse yalnız adaptör değişir.

Değişimlerden Korunma (Protected Variations): İleride değişebilecek, sayısı artıp azalabilecek veya çalışma şekli değişebilecek noktaları belirleyin. Bunların etrafında kararlı arayüzler oluşturun. Kapsülleme, veri gizleme ve çok şekillilik de bu prensibe dayanır.

Yabancılarla konuşma (Don't Talk to Strangers): Bir nesne yalnız tanıdıklarına mesaj göndermelidir. Tanıdıklar:

- Kendisi (`this`).
- Metodunun parametreleri.
- Kendi özellikleri (attribute) ve tuttuğu koleksiyonların elemanları.
- Kendi yarattığı nesneler.

Kötü örnek: `Money amount = sale.getPayment().getTenderedAmount();` Bu satırda tanınan `Sale` üzerinden tanınmayan `Payment`'ın metodu çağrılıyor. `Payment` değişirse bu kod da değişmek zorunda kalabilir. Doğrusu: `Sale`'e yeni bir sorumluluk verilir, ör. `sale.getTenderedAmountOfPayment()`. Böylece yalnız `Sale`'e bağımlı kalınır.

İlklendirme ve görünürlük

İlklendirme senaryosu: Hangi nesne ilk yaratılacak, `main` nerede olacak, listeler nasıl oluşturulup doldurulacak, nesneler birbirine nasıl bağlanacak? Bu, ayrı bir senaryo olarak yazılabilir ve genellikle tasarımın en sonunda belirlenir. Arayüz nesnelerine de hangi denetçiyle konuşacakları başta verilmelidir.

Örnek (`Store` yaratılınca):

1. `Store`, `ProductCatalog`'u yaratır.
   - 1.1: Katalog boş bir ürün tanımı listesi yaratır.
   - 1.2: `loadProductSpecs()` çağrılır. Döngüde her ürün tanımı yaratılır (1.2.1) ve listeye eklenir (1.2.2).
2. `Store`, `Register`'ı yaratır ve kataloğu ona parametre olarak verir. Böylece kasa kataloğu görebilir.

Görünürlük (visibility): B'nin A'ya görünür olması, A'nın B'ye erişip mesaj gönderebilmesi demektir.

- Özellik görünürlüğü (attribute): B, A'nın üyesidir. A yaşadıkça görünür.
- Parametre görünürlüğü: B, bir metodun parametresidir. Yalnız metot süresince geçerlidir. Bağımlılık daha az olduğu için tercih edilir.
- Yerel görünürlük: B, A'nın bir metodunda yerel değişkendir.
- Global görünürlük: B, global alanda bir üyedir.

Örnek: `enterItem(id, quantity)` içinde `spec = catalog.getSpecification(id)` ile ürün tanımı yerel bir değişkene alınır. Sonra `sale.makeLineItem(spec, quantity)` çağrılır. `SalesLineItem` yaratılırken `spec`'i kendi özelliğine atar; artık ona özellik görünürlüğüyle erişir.

Tasarım sınıf diyagramı ve kodlamaya geçiş

Bütün senaryolar etkileşim diyagramlarıyla tamamlanınca tasarım sınıf diyagramı çizilir. Sınıflar, özellikler, tipler, metotlar, parametreler ve erişim hakları gösterilir. İlişkiler yönlü çizilir. Kavramsal modelde olmayan yeni bağlantılar ortaya çıkabilir.

Slayttaki örnek diyagram:

- `Store` (address, name; `addSale()`): `ProductCatalog`'u kullanır (Uses), `Register`'ı barındırır (Houses), tamamlanan `Sale`'lerin kaydını tutar (Logs-completed).
- `Register` (`endSale()`, `enterItem()`, `makeNewSale()`, `makePayment()`): `ProductCatalog`'a bakar (Looks-in), `Sale`'i yakalar (Captures).
- `ProductCatalog` (`getSpecification()`): 1..* `ProductSpecification` içerir.
- `ProductSpecification`: description, price, itemID.
- `Sale` (date, isComplete, time; `becomeComplete()`, `makeLineItem()`, `makePayment()`, `getTotal()`): 1..* `SalesLineItem` içerir, bir `Payment` ile ödenir (Paid-by).
- `SalesLineItem` (quantity; `getSubtotal()`): bir `ProductSpecification` tarafından tanımlanır (Describes).
- `Payment`: amount.

Bağımlılıklar (dependency) kesikli çizgiyle gösterilebilir. Örnek: `Register`, katalogdan aldığı `ProductSpecification`'ı yerel değişkende kullandığı için ona bağımlıdır.

Kodlama:

- Sınıf tanımları tasarım sınıf diyagramından, metotlar etkileşim diyagramlarından çıkarılır.
- ArgoUML veya Visual Paradigm gibi CASE araçları diyagramdan kod iskeleti üretebilir. Geriye metotların iç mantığını yazmak kalır.
- En az bağımlı sınıftan başlanır. Her sınıf birim testine tabi tutulur.
- Test programı önceden de yazılabilir: test güdümlü geliştirme (test driven development).

Ders sonrası notlar

- **İşbirliği diyagramının adı:** UML 2'de işbirliği diyagramının (collaboration diagram) resmî adı iletişim diyagramıdır (communication diagram). Ardışıl diyagramın İngilizce adı sequence diagram'dır.
- **GRASP'ın açılımı:** Derste "General Responsibility Assignment Software Patterns" dendi. Larman'ın kitabında "Patterns" yerine "Principles" da kullanılır. İkisi de aynı 9 prensibi anlatır.
- **Görünürlük ile ilgili karışıklık:** Hoca, "metot parametresini yerel bir değişkene atarsanız özellik görünürlüğüne dönüşür" dedi. Doğrusu: parametreyi metodun içinde yerel değişkene atarsanız yerel görünürlük olur. Nesnenin bir özelliğine (attribute) atarsanız özellik görünürlüğü olur. Örnekte `SalesLineItem`, `spec`'i kendi özelliğine atadığı için özellik görünürlüğü oluşur.
- **Görünürlük türü sayısı:** Hoca "genel olarak iki çeşidi var" dedi (01:33:53), ama dört tür anlattı: özellik, parametre, yerel, global.
- **Kasaya katalog verilmesi:** Hoca gerekçe olarak fiyat sorgusunu gösterdi (01:33:14). Oysa alt toplamda fiyatı `SalesLineItem`, ürün tanımından alır. Kasanın kataloğu görmesi `enterItem` içinde ürün tanımını alabilmesi içindir.
- **`getTenderedAmount`:** Hoca "kalan miktar" dedi (01:28:18). Tendered amount, müşterinin verdiği tutardır; kalan tutar ya da para üstü değildir.

- **Denetçi ve yapay sınıf:** Hoca “Controller yapay bir sınıf” dedi (01:23:11). Bu her zaman geçerli değildir: Örnekteki denetçi `Register`, gerçek dünyadaki bir kavramdır. Yalnız bu iş için kurulan senaryo denetçileri yapay sınıf sayılabilir.
- **Yapay sınıf ve uyum:** Hoca yapay sınıfın önce “uyumu biraz bozduğunu” (01:06:00), sonra “uyumu arttırdığını” (01:20:53) söyledi. Pure Fabrication genel olarak iyi uyumu destekler: İlk sözdeki sorun, bütün sınıfların yapay sınıfa bağımlı olmasıdır. Bu yüzden yukarıdaki “uyumdan vazgeçme” örneği, veritabanı işlerinin ayrı ekibe verilmesi gerekçesiyle sınırlı okunmalıdır.
- **Yaratıcı kalıbının şartları:** Derste “içeriyorsa, kaydını tutuyorsa ve kullanıyorsa ve başlangıç verilerine sahipse” diye söylendi. Bu şartların hepsinin birlikte sağlanması gerekmez. Biri bile yeterlidir. En güçlü aday, A’yı içeren veya birleştiren (aggregate) sınıftır.
- **Altyazıdaki bozuk terimler:** “wireless UML” = Violet UML, “opet” = `opt`, “alete” = `alt`, “golf kalıpları” = GoF kalıpları, “grass block” = GRASP, “gut class” = god class, “SDLC sınıf” = “bir sınıf”, “artificial diagram” = ardışıl diyagram, “Mac Payment” = `makePayment`.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi daha verimli çalışabilirsiniz.